

**Algorytmy i Złożoność Obliczeniowa – zadanie
projektowe nr 1:**

**Badanie efektywności wybranych algorytmów
sortowanie ze względu na złożoność obliczeniową**

Paweł Łachut

Nr albumu: 281237

Prowadzący: Dr inż. Zbigniew Buchalski

Termin zajęć: Piątki nieparzyste, Gr 17, 11:15-13:00

Termin oddania: 25.04.2025

WPROWADZENIE

Celem projektu było przeprowadzenie analizy efektywności wybranych algorytmów sortowania pod kątem ich złożoności obliczeniowej oraz zbadanie ich wydajności w zależności od rozmiaru sortowanych tablic i rozkładu danych. Zaimplementowane zostały cztery algorytmy sortowania:

- **Insertion Sort (sortowanie przez wstawianie)**

- **Heap Sort (sortowanie przez kopcowanie)**

- **Shell Sort (sortowanie Shella)**

- **Quick Sort (sortowanie szybkie)**

W ramach projektu, zaimplementowano funkcje umożliwiające sortowanie tablic dynamicznych zawierających dane typu całkowitego (int) oraz zmiennoprzecinkowego (float). Program umożliwia także wczytywanie danych z pliku tekstowego, generowanie losowych tablic o zadanym rozmiarze oraz wyświetlanie tablic przed i po sortowaniu. Czas sortowania mierzono za pomocą funkcji umożliwiających dokładny pomiar w systemie Windows, a wyniki zostały uśrednione na podstawie wielokrotnych prób (100 testów) dla różnych rozmiarów tablic.

Projekt zakładał również testowanie wpływu różnych początkowych rozkładów danych na czas wykonania algorytmu:

- **tablica losowa**

- **tablica posortowana rosnąco**

- **tablica posortowana malejąco**

- **tablica posortowana częściowo (33% i 66% początkowych elementów)**

Wyniki zostały przedstawione w postaci wykresów, umożliwiających porównanie efektywności poszczególnych algorytmów w zależności od rozmiaru tablicy oraz rodzaju danych.

W projekcie uwzględniono także wpływ różnych strategii algorytmów na czas sortowania, w tym dobór odstępów w algorytmie Shella oraz wybór pivotu w algorytmie sortowania szybkiego. Dla algorytmu Shella przetestowano różne sekwencje odstępów, które mają istotny wpływ na jego złożoność obliczeniową. Natomiast w przypadku algorytmu szybkiego sortowania analizowano, jak różne metody wyboru pivotu (takie jak skrajny lewy, skrajny prawy, środkowy czy losowy) wpływają na efektywność algorytmu.

WSTĘP TEORETYCZNY – opisy poszczególnych algorytmów sortowania

Sortowanie przez wstawianie (Insertion Sort)

Zasada działania:

Sortowanie przez wstawianie działa w sposób podobny do sortowania kart w ręce. Przechodzi przez kolejne elementy tablicy, traktując każdy z nich jak nową kartę, i wstawia go w odpowiednie miejsce w już posortowanej części tablicy.

1. Algorytm zaczyna od drugiego elementu (indeks 1) i porównuje go z poprzednimi elementami.
2. Jeśli jest mniejszy, przesuwa większe elementy w prawo, by zrobić miejsce.
3. Wstawia aktualny element w odpowiednie miejsce.
4. Proces powtarza się do końca tablicy.

Złożoność obliczeniowa:

- **Najlepszy przypadek (tablica już posortowana):** $O(n)$
- **Średni przypadek:** $O(n^2)$
- **Najgorszy przypadek (tablica posortowana malejąco):** $O(n^2)$

Charakterystyka:

- Stabilny (nie zmienia kolejności elementów równych).
 - Działa w miejscu (nie wymaga dodatkowej pamięci).
 - Bardzo wydajny dla małych zbiorów danych.
 - Łatwy do zaimplementowania.
-

Sortowanie przez kopcowanie (Heap Sort)

Zasada działania:

Heap Sort opiera się na strukturze danych zwanej **kopcem** (ang. **heap**), najczęściej maksymalnym kopcem (ang. **max-heap**), gdzie każdy rodzic jest większy od swoich dzieci.

1. Najpierw tworzony jest kopiec z tablicy wejściowej (operacja "heapify").
2. Następnie największy element (korzeń kopca) zamieniany jest z ostatnim elementem tablicy.
3. Ostatni element zostaje "wyłączony" z kopca, a kopiec ponownie przywracany jest do porządku.
4. Proces powtarza się aż do całkowitego posortowania tablicy.

Złożoność obliczeniowa:

- **Najlepszy, średni i najgorszy przypadek:** $O(n \log n)$

Charakterystyka:

- Niestabilny.
- Działa w miejscu ($O(1)$ pamięci dodatkowej).
- Wydajny dla dużych zbiorów danych.
- Deterministyczny (niezależnie od rozkładu danych, zawsze $O(n \log n)$).

Sortowanie Shella (Shell Sort)

Zasada działania:

Shell Sort to ulepszona wersja sortowania przez wstawianie, która najpierw porównuje i sortuje elementy oddalone o określoną liczbę pozycji (tzw. **odstęp**, ang. **gap**), stopniowo zmniejszając ten odstęp aż do 1.

1. Wybierana jest sekwencja odstępów (np. Shell lub Frank-Lazarus).
2. Dla każdego odstepu wykonywane jest sortowanie przez wstawianie w ramach podtablicy z tym odstepem.
3. Proces kończy się, gdy $gap = 1$, czyli przeprowadzane jest klasyczne sortowanie przez wstawianie.

Złożoność obliczeniowa (zależna od sekwencji odstępów):

- **Shell (oryginalna sekwencja):** od $O(n^2)$ do $O(n^{3/2})$
- **Frank-Lazarus:** ok. $O(n^{3/2})$ do $O(n \log^2 n)$
- **Najlepszy przypadek:** $O(n \log n)$
- **Najgorszy przypadek:** $O(n^2)$

Charakterystyka:

- Niestabilny.
- Działa w miejscu.
- Dobry kompromis między prostotą a wydajnością.
- Wydajność zależy silnie od sekwencji odstępów.

Sortowanie szybkie (Quick Sort)

Zasada działania:

Quick Sort działa na zasadzie **dziel i zwyciężaj** (ang. **divide and conquer**). Dzieli tablicę na dwie części względem tzw. **pivota**, tak aby po lewej stronie znalazły się elementy mniejsze, a po prawej większe.

1. Wybierany jest element tablicy jako pivot (w zależności od strategii wyboru: pierwszy, ostatni, środkowy, losowy).
2. Tablica jest dzielona na dwie części względem pivota (partycjonowanie).
3. Proces rekurencyjnie powtarzany jest dla każdej części.

Złożoność obliczeniowa:

- **Najlepszy i średni przypadek:** $O(n \log n)$
- **Najgorszy przypadek** (zły wybór pivota, np. skrajny przy już posortowanej tablicy): $O(n^2)$

Charakterystyka:

- Niestabilny.
- Bardzo szybki w praktyce, zwłaszcza dla dużych zbiorów danych.
- Działa w miejscu (ale używa stosu rekurencyjnego).
- Wydajność silnie zależy od wyboru pivota

Zestawienie złożoności obliczeniowej i właściwości wybranych algorytmów sortowania

Algorytm	Najlepszy przypadek	Średni przypadek	Najgorszy przypadek	Pamięć dodatkowa	Stabilność
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	tak
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	nie
Shell Sort (Shell)	$O(n \log n)$	$O(n^{5/4}) - O(n^{3/2})$	$O(n^2)$	$O(1)$	nie
Shell Sort (Frank-Lazarus)	$O(n \log n)$	$\approx O(n (\log n)^2)$	$\approx O(n^{3/2})$	$O(1)$	nie
Quick Sort (dobry pivot)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	nie

PLAN EKSPERYMENTU

Implementacja:

- Projekt został zaimplementowany w języku C++, posługując się szablonami (templates), aby można było łatwo wykorzystać zaimplementowane algorytmy do sortowania tablic zawierających elementy różnych typów (int, float). Wszystkie sortowania są rosnące zgodnie z założeniami.

Pomiar czasu:

W celu dokonania precyzyjnych pomiarów czasu działania poszczególnych algorytmów sortowania, wykorzystano mechanizm zegara wysokiej rozdzielczości z biblioteki **chrono**, a dokładniej **std::chrono::high_resolution_clock**.

Pomiar był realizowany poprzez pobranie znacznika czasu tuż przed rozpoczęciem sortowania oraz bezpośrednio po jego zakończeniu. Następnie obliczana była różnica pomiędzy tymi dwoma punktami czasowymi, która została przeliczona na mikrosekundy i przekształcona do jednostki milisekund, zapewniając odpowiednią precyzję wyników.

Dodatkowo, aby zwiększyć wiarygodność i odporność na przypadkowe odchylenia wyników spowodowane czynnikami zewnętrznymi (np. działaniem innych procesów w tle), pomiar czasu wykonywano wielokrotnie dla tych samych parametrów testu. W

każdej iteracji generowano nową tablicę danych (według wybranego typu rozkładu), a wynik sortowania był mierzony oddzielnie. Ostateczny czas prezentowany w wynikach eksperymentu stanowił wartość średnią, obliczoną na podstawie sumy czasów uzyskanych w kolejnych próbach, podzieloną przez liczbę powtórzeń.

Ważnym założeniem implementacyjnym było to, że zarówno alokacja pamięci dla tablicy testowej, jak i jej inicjalizacja danymi wejściowymi (czyli generacja elementów) odbywały się przed rozpoczęciem pomiaru czasu. Dzięki temu wyniki odzwierciedlają wyłącznie czas potrzebny na wykonanie operacji sortowania, co pozwala na rzetelne porównanie efektywności różnych algorytmów przy identycznych warunkach wejściowych.

Czynniki mogące mieć wpływ na czas sortowania, które zostały poddane testom to:

Rozmiar sortowanej tablicy:

Pomiary przeprowadzono dla tablic liczb całkowitych (int) o 7 reprezentatywnych rozmiarach: 10000, 20000, 40000, 80000, 160000, 320000, 640000

Pojedynczy pomiar może być obarczony błędem, więc pomiary dla konkretnego rozmiaru tablicy zostały wykonane wielokrotnie (100 razy), za każdym razem generując nowy zestaw danych

Rozkład elementów w tablicy:

Początkowe ułożenie danych w tablicy może mieć wpływ na czas sortowania, więc rozpatrzono następujące przypadki:

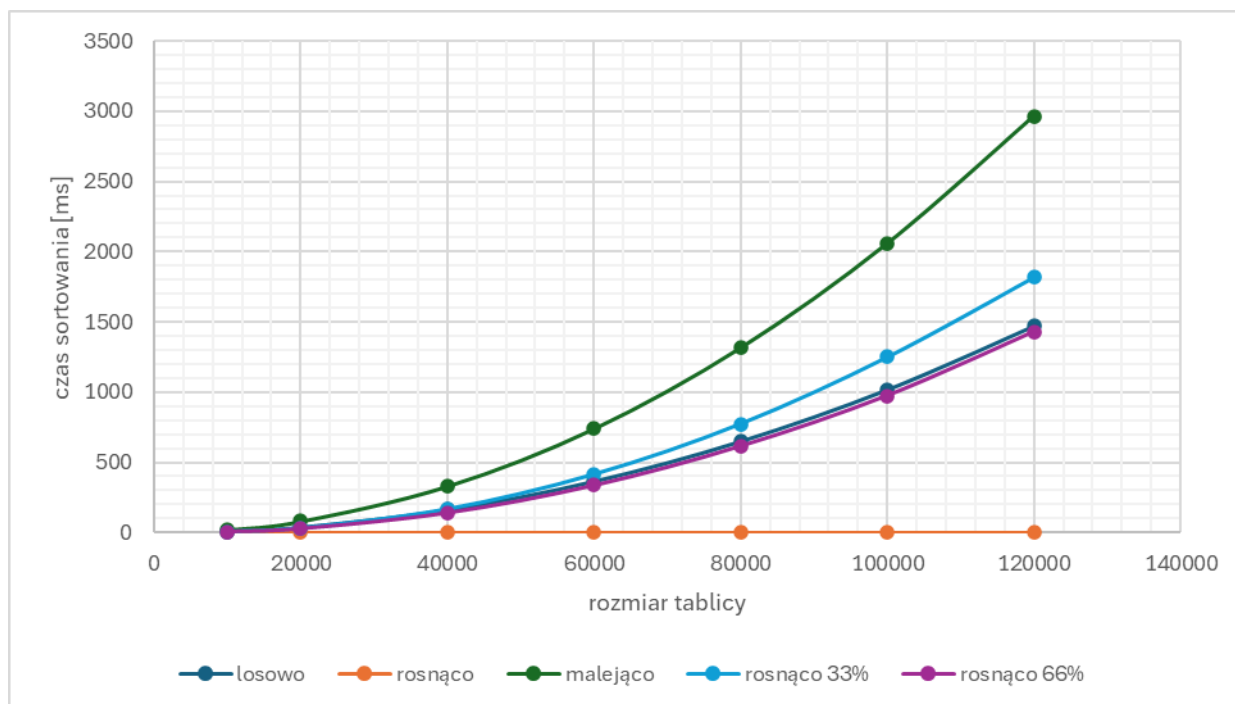
- tablica całkowicie losowa
- tablica posortowana rosnąco
- tablica posortowana malejąco
- tablica posortowana częściowo (33% i 66% początkowych elementów już posortowanych))

Typ danych w tablicy:

Badaniom podlegały tablice zawierające dane typu całkowitego (int) oraz zmiennoprzecinkowego (float). Dla obu typów danych przeprowadzono 100 testów na losowo generowanych tablicach zawierających 80000 elementów, a wyniki uśredniono.

PRZEBIEG EKSPERYMENTU

Sortowanie przez wstawianie:

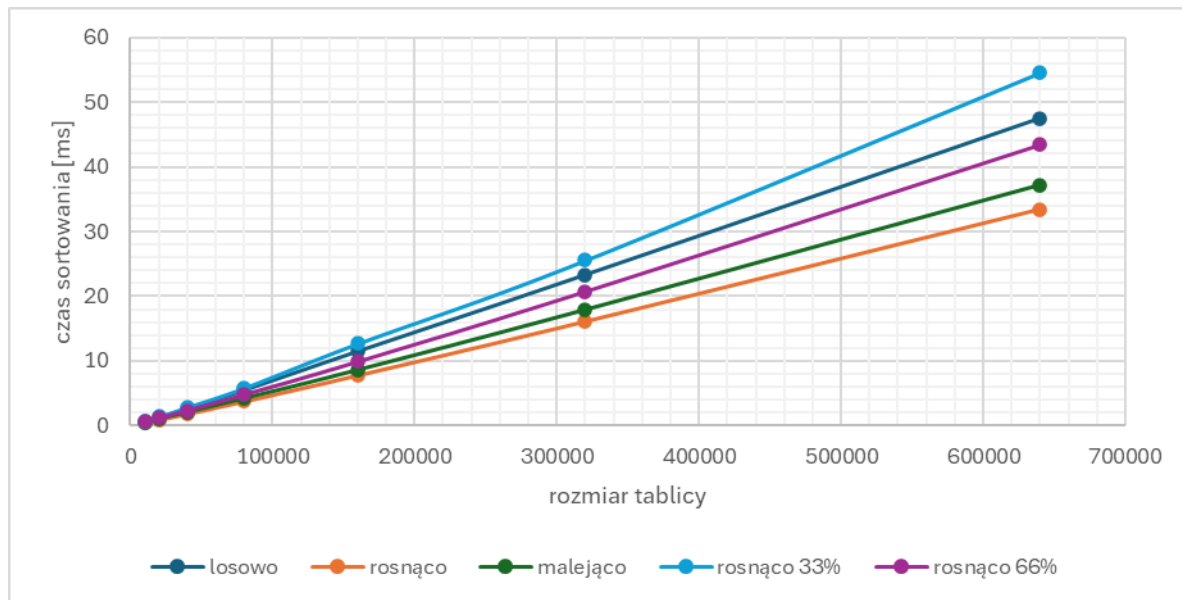


Wykres 1: Zależność czasu sortowania od rozmiaru tablicy, dla różnych rozkładów początkowych tablicy – sortowanie przez wstawianie

Wnioski:

- Dla algorytmu sortowania przez wstawianie zmniejszone zostały testowane rozmiary tablic ze względu na wolny czas wykonywania algorytmu.
- Kształt wykresu jest zbliżony do paraboli, co wynika z kwadratowej złożoności obliczeniowej sortowania przez wstawianie.
- Dla sortowania przez wstawianie najgorszym przypadkiem jest tablica posortowana malejąco, co pokrywa się z uzyskanymi wynikami przedstawionymi na wykresie. W algorytmie każdy nowy element musi przesunąć się aż do początku.
- Przypadkiem najlepszym jest tablica posortowana rosnąco, zamiast wielu porównań mamy tylko jedno na każdy element.

Sortowanie przez kopcowanie:

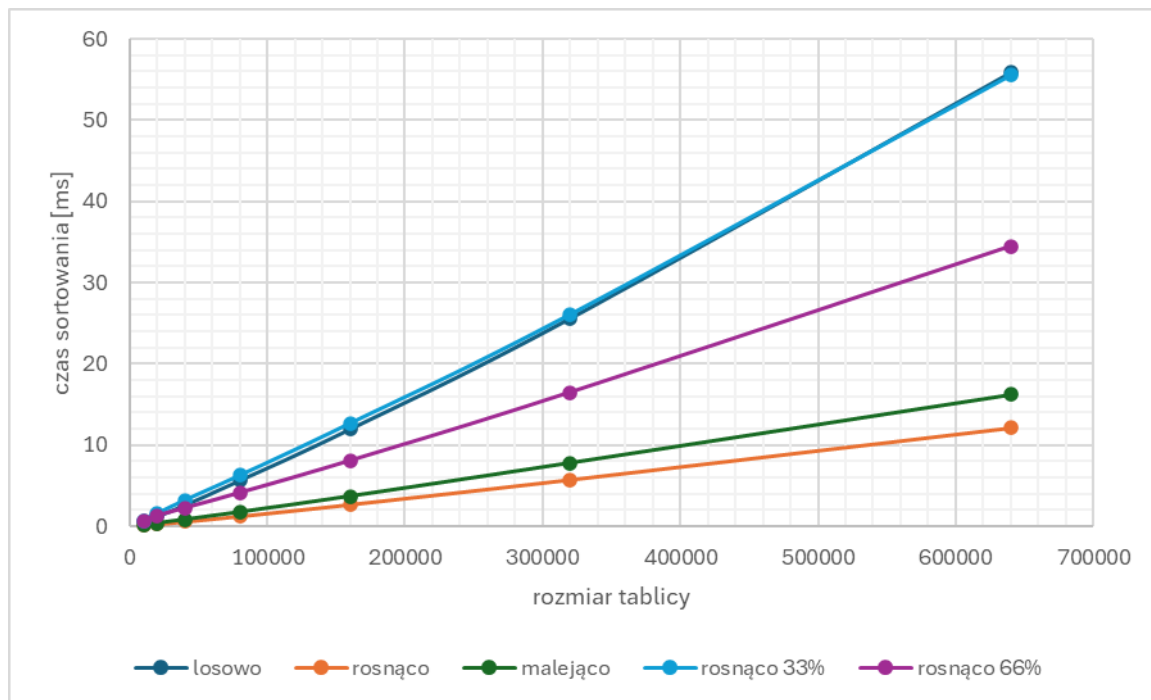


Wykres 2: Zależność czasu sortowania od rozmiaru tablicy, dla różnych rozkładów początkowych tablicy – sortowanie przez kopcowanie

Wnioski:

- Podwojenie rozmiaru tablicy powoduje wzrost czasu sortowania o około 2-2,5x, co zgadza się z teoretyczną złożonością $O(n \log n)$.
- Jak widać na wykresie, rozkład początkowy tablicy ma bardzo małe znaczenie w przypadku sortowania przez kopcowanie. Wpływ na czas ma głównie budowa kopca, dzielenie i scalanie czy rekurencyjne partycjonowanie. Pozostaje one wciąż proporcjonalny do $n \log n$, dlatego różnice wynikające z początkowego rozkładu są relatywnie małe.

Sortowanie Shella (Shell)

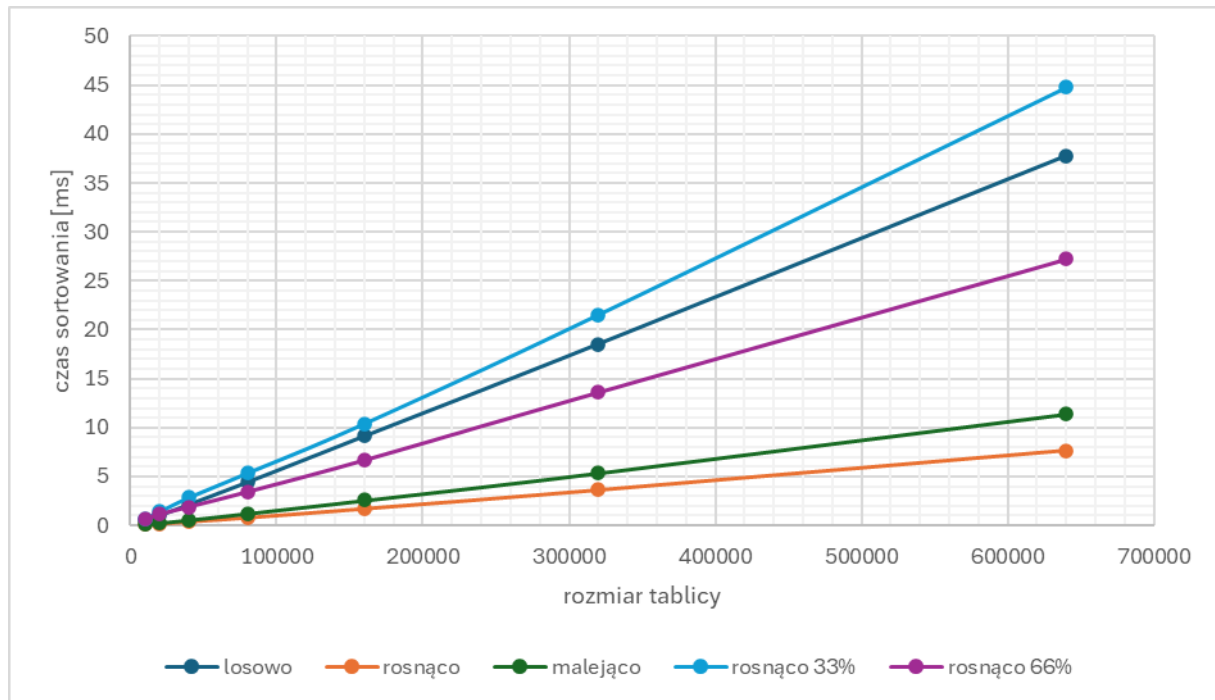


Wykres 3: Zależność czasu sortowania od rozmiaru tablicy, dla różnych rozkładów początkowych tablicy – sortowanie Shella (Shell)

Wnioski:

- Czas rośnie praktycznie proporcjonalnie do $n \log n$ (krzywa przybliża wykładniczy wzrost wolniejszy niż n^2), co potwierdza teoretyczną złożoność Shell Sort $\approx O(n \log^2 n)$ (dla sekwencji $n/2, n/4, \dots$).
- Dla wstępnie posortowanej rosnąco tablicy czasy są najniższe. W takim przypadku sortowanie Shella wykonuje minimalną liczbę przesunięć w każdym sortowaniu.
- Najgorszy przypadek występuje dla tablicy posortowanej w 33% i losowo posortowanej (są do siebie bardzo zbliżone). Wynika to z niekorzystnego rozmieszczenia elementów względem kolejnych kroków sortowania.

Sortowanie Shella (Frank & Lazarus)

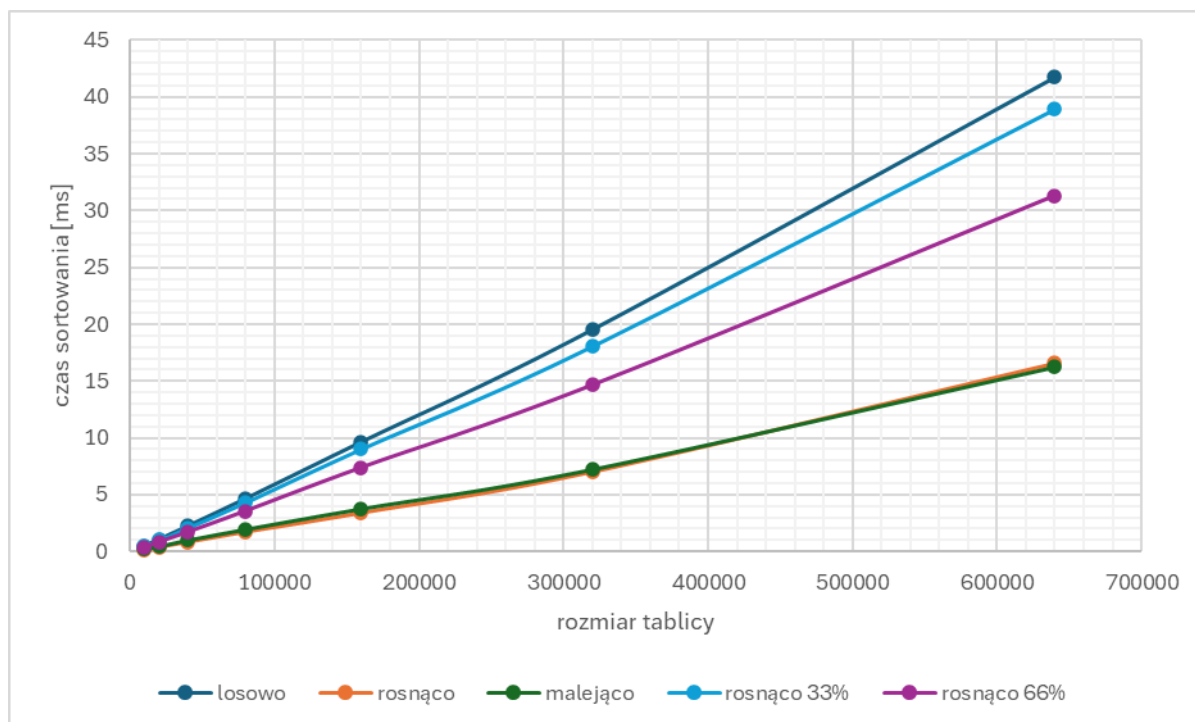


Wykres 4: Zależność czasu sortowania od rozmiaru tablicy, dla różnych rozkładów początkowych tablicy – sortowanie Shella (Frank & Lazarus)

Wnioski:

- rosną w sposób zbliżony do $O(n \log^2 n)$, ale są lepsze niż w wersji Shell'a, szczególnie dla danych uporządkowanych.
- Sortowanie Shella z ciągiem Franka i Lazarusa okazuje się wydajniejsze i bardziej stabilne niż jego klasyczna wersja. Szczególnie dobrze radzi sobie z dużymi danymi i przypadkami już uporządkowanymi. Dzięki lepszej sekwencji odstępów ogranicza liczbę kosztownych operacji, co znacząco poprawia czas działania w praktyce. Jest to dobra alternatywa dla sortowań $O(n \log n)$ w zastosowaniach, gdzie istotna jest prostota implementacji oraz względna wydajność bez korzystania z kopców czy rekurencji.

Sortowanie szybkie – środkowy pivot

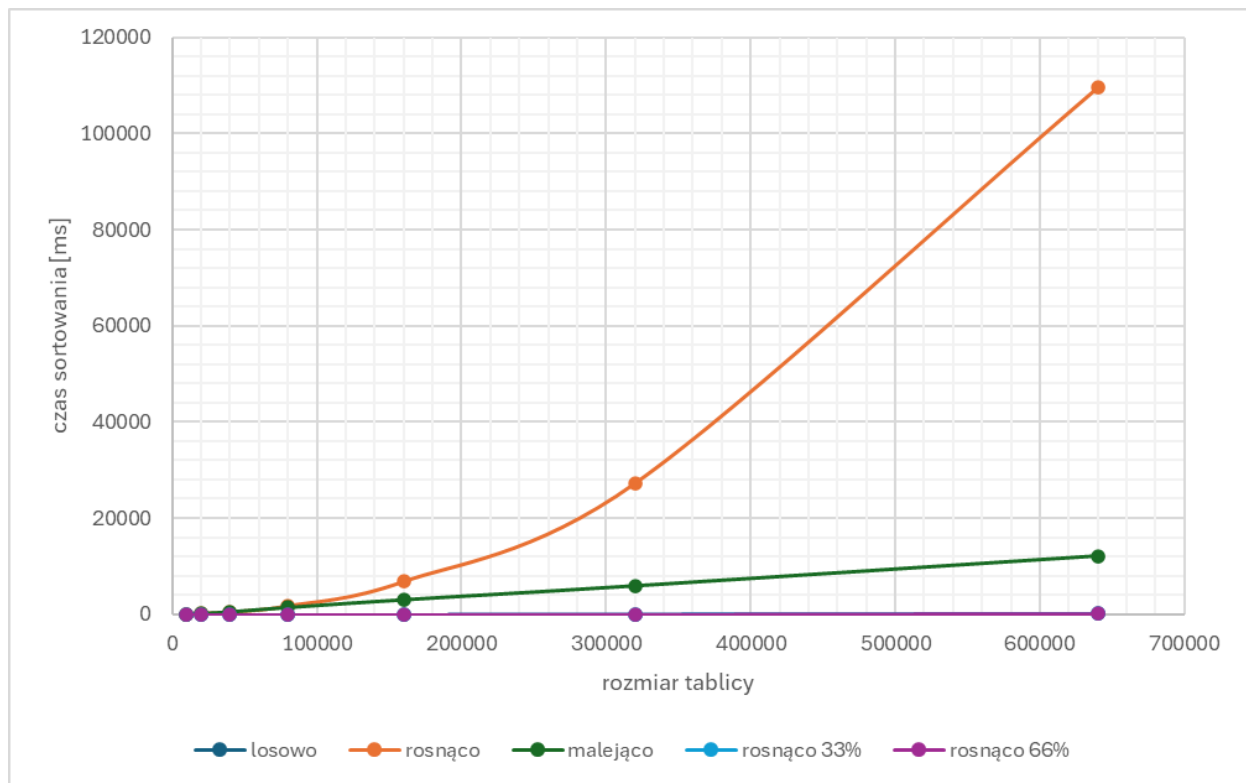


Wykres 5: Zależność czasu sortowania od rozmiaru tablicy, dla różnych rozkładów początkowych tablicy – sortowanie szybkie – środkowy pivot

Wnioski:

- Jak widać na wykresie, czas rośnie w przybliżeniu proporcjonalnie do $n \log n$, co zgadza się z przewidywaniami dla sortowania szybkiego dla optymalnego wyboru pivotu.
- Najniższe czasy osiągnięto dla posortowanych tablic. Oznacza to, że środkowy pivot dobrze radzi sobie, gdy dane są już uporządkowane.
- Różnice między rozkładami są relatywnie, co świadczy o stabilności wyboru środkowego pivotu.

Sortowanie szybkie – lewy pivot



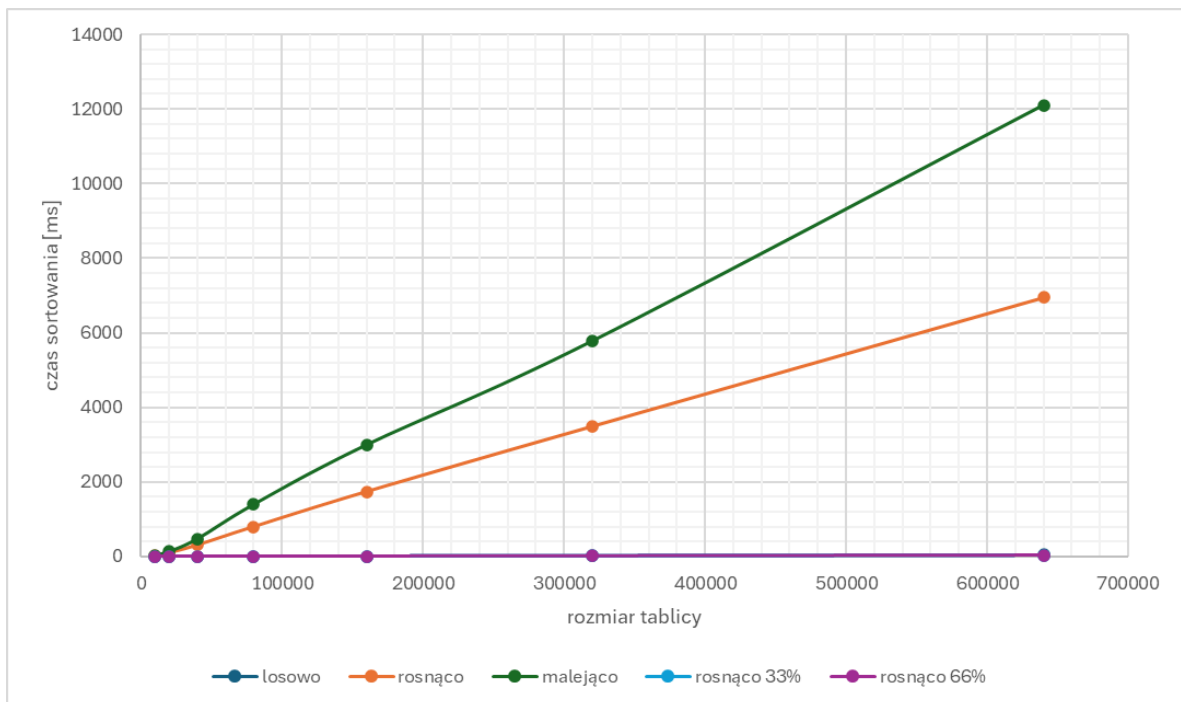
Wykres 6: Zależność czasu sortowania od rozmiaru tablicy, dla różnych rozkładów początkowych tablicy – sortowanie szybkie – lewy pivot

Wnioski:

- Wybór lewego pivotu prowadzi do degeneracji do kwadratowego czasu sortowania na tablicach już posortowanych (rosnąco lub malejąco). Choć dla danych losowych algorytm zachowuje złożoność $O(n \log n)$, w praktyce ryzyko wystąpienia przypadków bliskich $O(n^2)$ jest zbyt duże, dlatego pivot "lewy" jest niewskazany w większości implementacji algorytmu sortowania szybkiego.

- Nie widać znaczącej różnicy pomiędzy tablicami posortowanymi rosnąco w 33 % i 66% i losowo posortowanymi tablicami. Wynika z faktu, że dla lewego pivotu pierwszy element tablicy, czyli pivot zawsze pochodzi z posortowanej części. To powoduje powtarzalnie niekorzystny podział, niezależnie od tego, ile procent danych już jest uporządkowanych, co sprawia, że algorytm zachowuje się podobnie jak przy danych losowych.

Sortowanie szybkie – prawy pivot

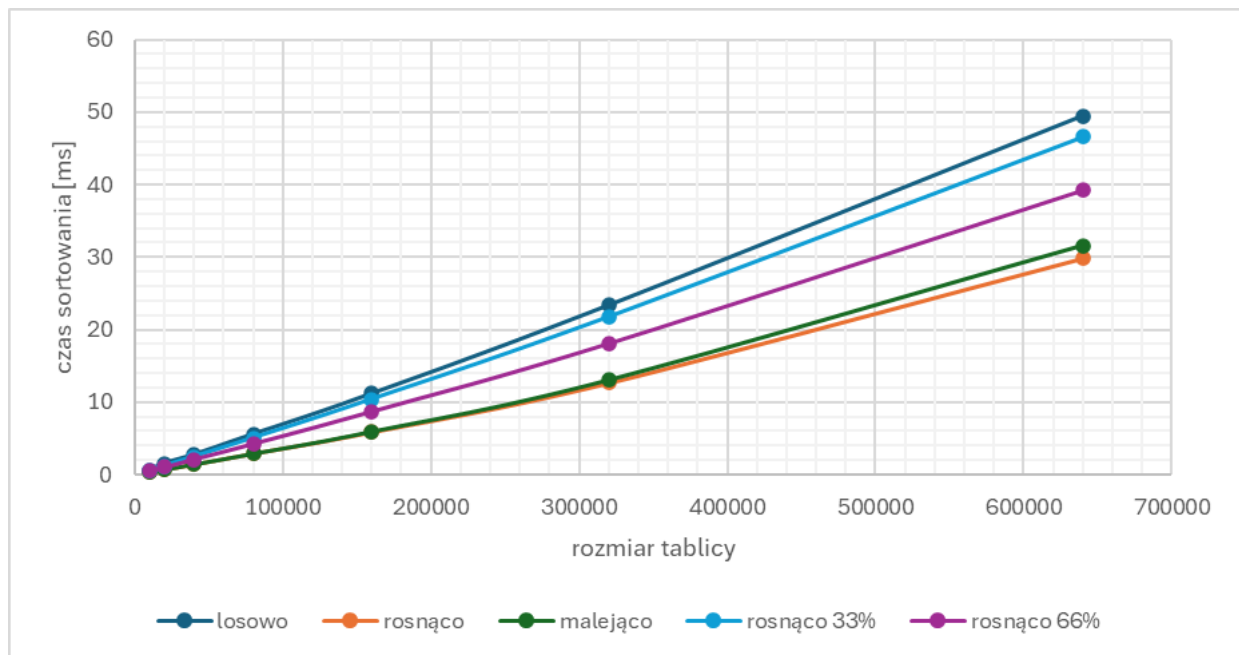


Wykres 7: Zależność czasu sortowania od rozmiaru tablicy, dla różnych rozkładów początkowych tablicy – sortowanie szybkie – prawy pivot

Wnioski:

- Podobnie jak przy wyborze pierwszego elementu jako pivotu, również ostatni element powoduje drastyczny wzrost czasu sortowania w przypadku danych posortowanych rosnąco lub malejąco. Pomimo dobrej wydajności dla danych losowych, ten wariant również może ulec katastrofalnej degeneracji do złożoności $O(n^2)$.
- Dla rozkładu „rosnąco 33%” oraz „rosnąco 66%”, czasy są zbliżone do danych losowych. Algorytm w tych przypadkach nie wpada jeszcze w degenerację, ponieważ końcowy element nie zawsze znajduje się w najbardziej uporządkowanym miejscu.

Sortowanie szybkie – losowy pivot



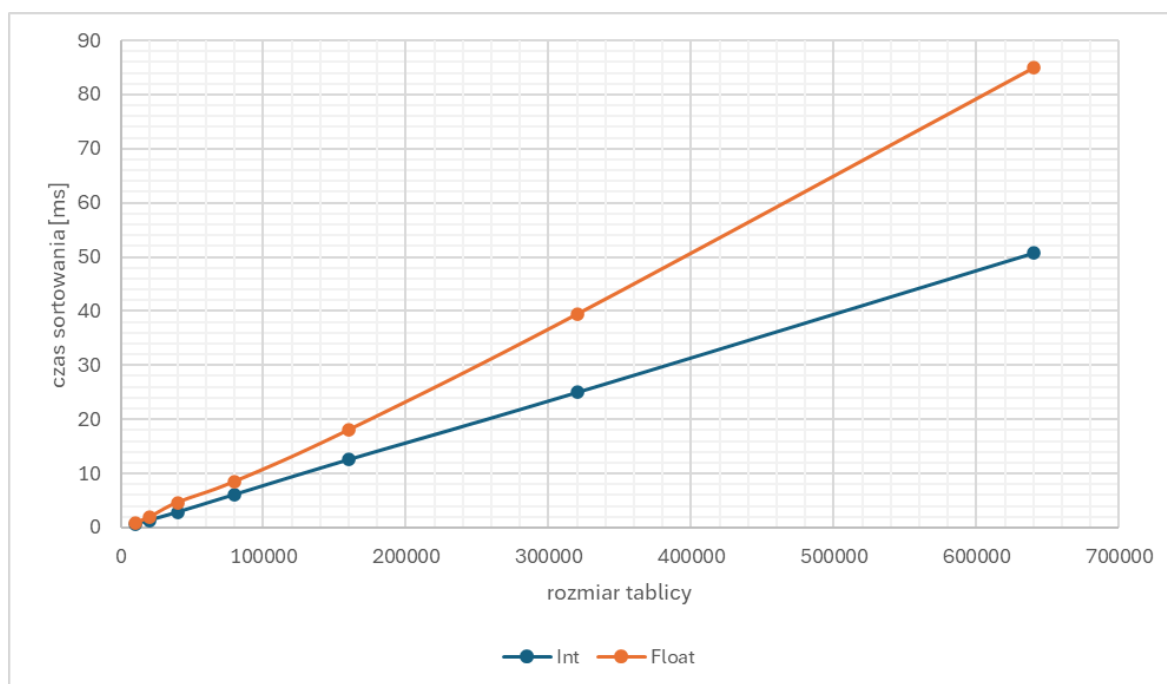
Wykres 8: Zależność czasu sortowania od rozmiaru tablicy, dla różnych rozkładów początkowych tablicy – sortowanie szybkie – losowy pivot

Wnioski:

- W przeciwieństwie do lewego czy prawego pivota, czas sortowania nie zależy drastycznie od początkowego ułożenia danych.
- Dane rosnące i malejące nie powodują znaczącego spowolnienia – różnice są niewielkie, maksymalnie 25–35% względem przypadku losowego.
- Czas wykonania rośnie proporcjonalnie do rozmiaru danych - sugeruje to złożoność $O(n \log n)$, która jest teoretycznie oczekiwaną wartością średnią dla sortowania szybkiego z losowym pivotem.

Badanie wpływu danych (int i float) na czas sortowania

Do analizy wpływu typu danych na czas sortowania zostało użyte sortowanie przez kopcowanie, ze względu na niewielki wpływ rozkładu początkowego tablicy na czas wykonywania sortowania. Dzięki takiemu zabiegowi nie trzeba testować tak wielu przypadków. Badane tablice były generowane losowo.



Wykres 9: Zależność czasu sortowania od rozmiaru tablicy, dla różnych typów danych (int i float) – sortowanie przez kopcowanie

Wnioski:

- Dla obu typów danych czasy rosną zgodnie z oczekiwaniami $O(n \log n)$
- Operacje na typie float zajmują jednak widocznie więcej czasu. Wymagają one użycia jednostki zmiennoprzecinkowej (FPU), która działa wolniej niż jednostka arytmetyczno-logiczna (ALU) używana w int. Dodatkowo porównania i operacje arytmetyczne na float zajmują więcej cykli procesora niż te same operacje na int. Dane float mogą też zajmować więcej miejsca w pamięci i być gorzej dopasowane do cache procesora, co obniża wydajność przy dużych tablicach

Wnioski końcowe:

Zrealizowany projekt pozwolił na praktyczne zbadanie efektywności wybranych algorytmów sortowania w różnych warunkach wejściowych i przy różnym doborze parametrów. Poniżej przedstawiono najważniejsze obserwacje i wnioski:

- **Zgodność z teorią** – wyniki uzyskane w eksperymentach potwierdzają teoretyczne złożoności obliczeniowe algorytmów. Algorytmy o złożoności kwadratowej (jak Insertion Sort) wykazują znacznie gorszą skalowalność niż algorytmy o złożoności logarytmicznej lub logarytmiczno-liniowej (Heap Sort, Quick Sort, Shell Sort).

- **Wpływ rozkładu danych** – rozkład początkowy tablicy istotnie wpływa na czas działania niektórych algorytmów, szczególnie Insertion Sort i Quick Sort. W przypadku Heap Sort i zaawansowanych wariantów Shell Sort wpływ ten jest dużo mniejszy.

- **Rola strategii w algorytmach** – zastosowanie różnych sekwencji odstępów w Shell Sort (np. Frank-Lazarus zamiast klasycznego ciągu Shella) oraz różnych metod wyboru pivotu w Quick Sort znacząco wpływa na praktyczną wydajność algorytmów. Wybór odpowiedniej strategii może znacząco obniżyć czas sortowania.

- **Porównanie typów danych** – sortowanie danych typu float jest generalnie wolniejsze niż int, co wynika z większej złożoności operacji zmiennoprzecinkowych oraz często bardziej złożonych porównań na poziomie procesora.

- **Przydatność sortowań** – dla dużych danych i zastosowań ogólnych najlepsze rezultaty daje Quick Sort z dobrze dobranym pivotem oraz Shell Sort z sekwencją Franka i Lazarusza. Insertion Sort, mimo niskiej wydajności przy dużych zbiorach, pozostaje przydatny dla bardzo małych tablic lub fragmentów danych.

Literatura:

Cormen T., Leiserson C.E., Rivest R.L., Stein C., Wprowadzenie do algorytmów, WNT

Serwis Edukacyjny w I-LO w Tarnowie – strona internetowa:

https://eduinf.waw.pl/inf/alg/003_sort/0004.php